

Part II — Practical Understanding

IT Asset Manager · CHUO JESSE Source code: <https://github.com/Jessechuo/sophicinterview>

Answers to Q3, Q4 and Q5. (Q1 ERD and Q2 system process flow are supplied as separate diagram files.)

Q3. Error handling strategy

Errors are handled in one place on each side. The API throws typed exceptions that a global handler converts into RFC 7807 ProblemDetails responses, so every failure has the same shape and the frontend can react to the status code instead of parsing messages. The browser never sees a stack trace.

1. Invalid user inputs

- **In the browser:** each form validates before it submits (required fields, formats, lengths, a purchase date that can't be in the future) and shows the message under the field it belongs to.
- **In the API:** the same request is validated again with data annotations, because the API is also reachable directly. A failure returns **400** with a `ValidationProblemDetails` body listing each invalid field.
- **Back in the browser:** those per-field messages are mapped onto the matching inputs, so a rule that only the server knows about still lands on the right field rather than in a generic banner.
- Client validation is for speed of feedback only. The server is the authority.

2. Asset update errors

Situation	Response	What the user sees
Asset no longer exists	404	A message that the record was not found, and the list reloads
Duplicate asset tag or serial number	409	An error under that field, naming the conflict
Someone else edited the asset first	409	A notice that the asset changed, and the form reloads with the current values so no edit is silently overwritten
A rule is broken, e.g. deleting or retiring an asset that is still assigned	409	A toast explaining why, e.g. "unassign it first"
Unexpected failure	500	A generic message with a trace id; the details are logged server-side only

Concurrency is enforced with PostgreSQL's row version (`xmin`), which the client sends back with the edit. Deletes are soft deletes, so an asset's history stays intact for the activity log.

3. Unauthorized access attempts

Authorization is enforced on the server and merely reflected in the UI, never the other way round.

- **Not signed in, or an expired token:** 401. The app clears the session and returns to the login page with a note that the session expired.
- **Signed in but not permitted** (a normal user trying an admin action): 403. Admin-only endpoints are protected by role checks, and the default policy requires authentication, so a new endpoint is protected unless it explicitly opts out.
- **Ownership rules on top of roles:** a normal user only ever sees the tickets they raised, in the list and when opening one by id, so a valid token for someone else's ticket is still refused.
- **In the UI:** admin-only buttons and navigation are hidden, and admin-only pages redirect to a "no access" screen. This is convenience, not security — the same request sent directly to the API is rejected with the same 403.

The role matrix is covered by integration tests: every endpoint is called as an admin, as a normal user and with no token at all, and the expected 200/401/403 is asserted.

Q4. Features not implemented

Every must-have, nice-to-have and extra-challenge feature in the brief is implemented. The items below were considered, left out on purpose, and are the ones I would pick up next.

1. Password reset by email. Deprioritized because it needs an email service and token storage, and the assessment's accounts are fixed demo logins; today an administrator sets a new password, which the "forgot password" link explains. *Given more time:* a single-use, time-limited reset token stored as a hash with an expiry, emailed through a provider such as SendGrid, plus rate limiting on the request endpoint.

2. File attachments on tickets and assets (photos of damage, invoices, warranty PDFs). Deprioritized because storing files well means object storage, size and type limits and virus scanning — a lot of surface area for a feature the brief didn't ask for. *Given more time:* upload straight to object storage (S3 or Azure Blob) with a pre-signed URL, and keep only the metadata and a reference in the database.

3. Bulk import of assets from Excel. Deprioritized because export was the stated requirement, and a safe import needs a preview, per-row validation and a way to undo a bad file. *Given more time:* reuse the existing ClosedXML setup to parse the sheet, validate every row into the same DTOs the API already uses, show a preview of accepted and rejected rows, then commit the accepted ones in one transaction.

4. Notifications. Deprioritized as it depends on email delivery and on people's notification preferences. *Given more time:* a background job (Hangfire or a hosted service) that emails the requester when a ticket's status changes and warns an administrator about assets overdue for maintenance.

5. Activity logging for user administration. The activity log covers assets only — create, update, delete, assign and unassign — which is what the brief specified. *Given more time:* I would log user and role changes to the same table, since the entity type and entity id columns already allow it, and add a filter so the two kinds of history can be read separately.

Q5. Debugging scenario

The scenario: the asset list works for admins but returns 500 for normal users, only on the test/staging server, and everything is fine locally.

The facts narrow it down before any code is read: it is role-specific, environment-specific, and a 500 means the server threw, not that authorization refused (that would be a 403). So the code path that runs *only* for a normal user is hitting something that differs between my machine and staging — configuration, data, or the deployed build.

Step 1 — Reproduce it deliberately

Sign in to staging with a normal user's account and open the asset list. Then call the same endpoint directly (Swagger, curl or Postman) with that user's token, which separates an API fault from a UI fault. Confirm the admin account still works, and check whether every normal user fails or only some, since "only some" points at their data rather than at the code.

Step 2 — Get the real exception

The response is a ProblemDetails body with a **trace id**. Searching the server logs for that id gives the exact exception and stack trace, which the browser deliberately never shows. Tools: the host's log stream (Render/Azure/docker logs), or a log search such as Seq or Application Insights. This usually identifies the failing line straight away; the remaining steps are for confirming *why* it differs on staging.

Step 3 — Compare the three things that differ between environments

Suspect	How I check it	Typical root cause
Configuration	Compare the staging environment variables against the app's settings	A missing or differently-named setting that only the non-admin path reads
Identity and claims	Decode the user's JWT (jwt.io) and compare the role claim with what the API expects	Roles arriving under a different claim type after deployment, so the user reaches code that assumes a role it can't find
Data and schema	Run the same query against the staging database with psql; check the migration history table is up to date	A migration not applied on staging, or staging data with a null where local seed data always had a value — e.g. a user with no department, or an asset whose assignee row was deleted

In this codebase the asset list itself is not filtered by role — both roles run the same query — so a failure for one role only points at the identity behind the request rather than at the listing logic. The non-admin path is the one that reads the user's id from the token (the admin path skips it, as in the ticket list, where non-admins are filtered to their own records). If that claim is absent or has a different name in the deployed configuration, the code that parses it throws for normal users while admins sail past. A missing migration on staging produces the same signature whenever the extra column is touched on only one of the two paths.

Step 4 — Fix it, then prove the fix

1. **Write a failing test first**, at the level of the cause: an integration test that calls the asset list as a normal user, with exactly the state that broke (no department, a deleted assignee, a token without the claim). It should fail with the same 500 before the fix.

2. **Fix the cause, not the symptom.** A try/catch that hides the exception would turn a 500 into an empty list and a silent bug.
3. **Run the whole suite** (106 integration tests), which includes calling every endpoint as an admin, as a normal user and unauthenticated — this is how I confirm the other roles still behave.
4. **Verify on staging itself:** redeploy, then repeat step 1 as a normal user *and* as an admin, and check the logs show no new exceptions.
5. **Close the gap that let it through:** if the cause was configuration, add a startup check that fails fast with a clear message when a required setting is missing, so the next occurrence is obvious at deploy time instead of at 500-time.

Tools used: browser DevTools (network tab and the failing response), Swagger/curl for calling the API directly, the host's log stream searched by trace id, jwt.io for the token's claims, psql for the staging data and migration history, EF Core query logging, and dotnet test for the regression suite.